

Stemma Session Summary

Consolidated — through 17 September 2026

19 September 2026

Recap — June & July 2026

Architecture: per-DAW renderers in separate repos, coordinated by a daemon through a shared versioned contract; communication only via the job folder. Walking skeleton proven with a stubbed Ableton export. **5 July**: the pipeline became real for Logic. **22 July**: DialogGuard rule C10; the stemma library UI replaced the one-shot app; branch renamed logic-slice → logic-and-UI. **23 July**: the Stemma folder — a watched Finder folder that IS the library; aliases, never copies; droplet; Finder-alias support.

Recap — 24 August 2026

Root cause of a month of failures: **macOS Accessibility permission is inherited from the launching app** — Terminal.app needed the grant. Driver hardening: 10-minute ready ceiling, retryable export-dialog stage. Open case opened: the missing “Sum 8” summing stack.

Recap — 7 September 2026

The staged-success (“checkpoint”) model: outcomes ok / ok_warnings / failed with the border after the raw-differs verify (owner’s call: stems are the deliverable, packaging is bonus). Raw-differs downgraded from a false-failure to an honest warning (no-FX projects ship). Robust cleanup (.DS_Store scrub). The key insight that four roadmap items (pre-flight warning, completeness check, silent-stem detection, effects check) are ONE feature: the missing `logicx_parser.py`, Logic’s twin of Ableton’s `als_parser.py`. Busy-click patience (40 s). Date-audit: summaries are now dated from the system clock, never from conversation memory.

Recap — 10 September 2026

Sum 8 filed as an unreproduced ghost. Naming untangled: “Stemma” the repo (Noe’s — old app + the Ableton kitchen) vs “stemma” the product (the logic-renderer repo — current app + Logic kitchen). Ableton integration plan of record: **a second backend on its own port, routed by file type; the two backends’ APIs are twins**. Live-warnings design agreed (raise warnings as they become knowable; a mid-render warning is only actionable with a cancel button, so cancel came first). The cancel button built and debugged live to a 1-second abort: Cmd-period posted straight into Logic’s process with Quartz, bounce-wait scans capped at 4 s (System

Events serialises Apple events and jams for 16-19 s against a pegged Logic), DialogGuard rule C11 `disk_too_slow`.

Recap — 14 September 2026

Live warnings layers 1+2 built (pushed into the shared state as discovered; silent-stem scan, pass symmetry, completeness). The “missing track” ghost root-caused: **a lit solo** silences stack/DMD tracks and Trim Silence erases the file; fix = read mute/solo from the track headers, clear solo with Option+S (only a System Events keystroke with Logic frontmost works), exclude muted stems. Logic’s export ignores mute, honours solo, skips region-less tracks. Cancel made ~1 s. Warning UI (green status + amber mark, hover card, click-to-pin cascade). Status column widened and left-aligned. Light lavender retheme. DAW badges drawn in the palette: Logic platter, “Live” wordmark, FL mango stencil traced from the real icon. Recon of the Ableton renderer and the owner’s sequencing: connect it as-is first, parity later. PR #2 opened (logic-and-UI into main).

Recap — 15 September 2026

The Ableton chapter opened: Stemma’s Flask server takes `STEMEXPORT_PORT`; the app spawns it on 5124 beside the Logic backend through one shared `spawnBackend()`, routes `.als` rows to it, and the health pill names whichever backend is down. **First full render at 20:35** (17 tracks, 18/15/18 stems, 2.0 GB zip). Four driver bugs fixed on the way: return tracks matched by the typed name instead of Live’s exported “A-Reverb” (prefer `EffectiveName`); a dialog scan that overran 8 s was fatal (now a skipped tick); a 5-minute export ceiling expired mid-pass while Live was still writing (now 30 min like Logic, honest error if reached); the backend’s output was buffered and it emitted no failure marker (unbuffered, marker, reason shown on the row). Fresh output folder per render on both DAWs (`<name> (2)`). Repos renamed: **Stemma** → **ableton-renderer** (owner’s FleapStems account; the app and Daemon try both names), **SendStems** → **Steminem** locally (aliases, app paths, venv and Claude memory fixed up). The checkout moved to the headless-V1 driver on `ableton-exporter-v1`; `SLice` superseded. Architecture schematic and a collaborator handoff page published; typeface pairings explored (no decision). Lesson: renames in Live reach the export only once the set is saved.

Recap — 16 September 2026

The parity chapter, complete. Owner’s goal: “both renderers provide the same experience.” (1) Two sets, not three: `01_With_FX` + `02_Raw` (`_raw` suffix); the returns-and-master pass kept as an opt-in flag, renders ~35% shorter. (2) Native sample rate, 24-bit, no dither: Live stores no per-set rate in the `.als`, so the engine rate is read from Live’s own `Log.txt` after launch (the old max-clip-rate guess gave 96 kHz on a 48 kHz set); stems 119 → 45 MB, zip 2.0 → 0.5 GB. (3) Never-save was already law in the headless driver; new gentle teardown clicks Cancel on Live’s export progress window before the never-save quit. (4) Cancel: `/export/cancel`, cooperative flag at every gate and inside the waits, immediate abort of Live’s render from the route’s thread, status `cancelled`, folder removed, row back to Ready — “perfect” mid-pass-1 and mid-pass-2. (5) Live warnings the Logic way (owner’s call: a missing stem is a warning, not

a failure): Solo, Muted (stems left out, named), Empty (Live writes a silent stem for a clip-less track; removed and named — both DAWs now say “Empty”), Silent, Missing media, Identical, Mismatch, Zip/Cleanup; staged success; one inbox note per render; verified with a “messy” set (2 solos, 4 mutes, 1 empty track). Found on the way: a dropped import crashed the server’s *finally* (leaked the sleep guard, never released the in-flight flag) — hardened. App: warnings clickable mid-render (delegated listener — the status cell is rebuilt every poll), health pill “initiating...” vs “offline”. Claude Code hooks: reply-done and needs-attention sounds. Lessons: import the module after an edit, not just compile it; Live writes silent stems where Logic writes nothing; a process-exists “wait ready” is blind to dialogs. Summary rule set: one recap per past session kept forever, full chapter for the newest day, Markdown source on disk.

Session of 17 September 2026 — packaging, layer 1: a shareable build

Owner’s question at the start: “what would it take to create a sharable prototype?” Answer in three layers — the bundle, first-run on a stranger’s Mac, distribution — and the bundle was built the same morning. Two decisions on the way: the Logic build had **never** been made in this repo (no `dist/` anywhere; the recipe was copied from the old app on 6 June and still bundled *librosa* and the *Anthropic SDK*), and the owner’s standing rule “**push always, in any repo**” now overrides *ableton-renderer*’s old “user pushes manually” convention.

The bundle

- `logic-renderer/build.sh` does it all: freezes both backends with `PyInstaller` (`logic-server`, `ableton-server`), gathers the Remote Script, packs the Electron app, produces `dist/stemma-1.0.0-arm64.dmg` (~124 MB). Product renamed **stemma** / `com.stemma.app`, the platter as a placeholder icon, documented in `docs/BUILD.md`.
- Both `build_python.py` recipes rewritten lean (hidden imports only for what is loaded lazily: `PyYAML`, `pyobjc`, `numpy` + `soundfile` for the Ableton silence check; `dialog_rules.yaml` bundled; the AI-renamer stack excluded). Logic’s `dialog_guard.py` got the `sys._MEIPASS` rules-path resolver backported from the Ableton engine.
- `main.js` gained the packaged branch for the Ableton backend, so both DAWs ship.

Three things that bit, all now built into the recipe

1. **Architecture.** The `python3` on `PATH` is Anaconda’s, which is `x86_64`: freezing with it put Intel binaries inside an Apple-Silicon app. A dedicated `arm64` build environment (`working/build-venv`, Homebrew Python 3.13) now freezes them.
2. **Signature.** Without a Developer ID, `electron-builder` leaves the bundle carrying Electron’s own, now-invalid seal, and Apple Silicon refuses to run it — no dialog, no log, the process just exits. The build now ad-hoc signs the whole bundle (`codesign --force --deep --sign -`) between `electron-builder`’s `--dir` and `--prepackaged` steps; verified valid inside the `dmg`.
3. **A red herring that cost an hour:** VS Code sets `ELECTRON_RUN_AS_NODE=1` in Claude’s terminal, which makes any Electron binary behave like plain Node and exit 0 silently. (Also: `asar extract-file ... package.json` writes into the current directory and clobbered the repo’s `package.json` once — restored.)

First tests on the owner's Mac

- “Failed to fetch” on Render: the one-file backends took ~20 s to unpack on every launch and the Render button never waited. Fixed twice over: backends frozen as folder bundles (`--onedir`, both answer ~4 s after launch), and Render/Render All now stay disabled until that DAW’s backend answers a health check (tooltip “renderer is starting — one moment”); a connection failure reads “renderer did not answer — it may still be starting, or it stopped”.
- Logic render stuck on “Launching Logic Pro...” with the audio-interface alert open: the packaged app is a new identity to macOS with no Accessibility grant, so the driver could not see Logic’s windows and the DialogGuard could not dismiss the alert it handles routinely in dev. macOS does not prompt for this on its own. New `permissions.py` (identical in both backends) runs before the DAW is launched: `AXIsProcessTrustedWithOptions` with the prompt option (macOS shows its own “would like to control this computer” dialog), then a System Events probe for the Automation grant; a missing grant fails the render at once with a message naming the exact System Settings switch, via the normal notification + inbox path. **Retest of this build still pending.**

Explained along the way (owner questions)

`main.js` is the Electron main process — the backstage half with Mac access (spawns the backends, notifications, inbox, folder watching) — versus `renderer/app.js`, the on-stage web page; why stemma is a web page in a window (Electron: Slack, VS Code, Discord, Figma work the same way; one UI for macOS/Windows, fast iteration, the DAW driving is Python anyway; the cost is ~200 MB and memory); and why the packaged app needs its own Accessibility and Automation grants for both DAWs (permission follows the responsible app — Terminal in dev, stemma when packaged; Ableton’s Remote Script socket needs none).

Git

All pushed: `logic-renderer f898e53, 73fed7c, 44ed8a0` on `logic-and-UI`; `ableton-renderer e5df618, 35cb5b5, d75f8f0` on the new `ableton-exporter-v4`.

Next steps

1. **Retest the dmg** with the permissions pre-flight (trash the old copy in Applications; only one stemma running). Expected: the Accessibility dialog, grant, possibly one relaunch, then a render past “Launching Logic Pro...”.
2. **Packaging layer 2 — first run on a stranger’s Mac:** port the Remote Script installer from the old Stemma shell (`installRemoteScript / getAbletonUserLibraryPath`, version-stamped) into the app; “Logic / Live not installed” shown honestly; default output folder on the internal disk; the sleep guard surviving an app quit. Test on a fresh macOS user account (no Python, no grants, licences still valid), then on a real second Mac.
3. **Layer 3 — distribution:** Developer ID (€99/year) + notarization; electron-builder does both once the certificate is in the Keychain. Until then: right-click → Open for trusted testers.
4. **Render-complete sound** in the app (and a failure sound; owner supplies files).
5. **Export-format option** per render: sample rate, bit depth, later the returns-and-master set.
6. Shared roadmap as before: staging-folder redesign for Logic, pre-flight “Render anyway /

Cancel", carry-overs (per-project output default, "-" stem prefix, Range decision, daemon inbox return, zip-only job folders, absorb daemon, collaborator notice).